

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

— * —



BÁO CÁO BÀI TẬP

BTVN 04

Họ Và Tên: Nguyễn Hoàng Long

Mã Số Sinh Viên: 20235367

Học Phần: IT4060

Mã Lớp: 168503

B01 : FTP_CLIENT

Ảnh minh họa chạy client:

```
sakakiyuya@DESKTOP-7E0C4GH:~/BTVN: 20235367/btvn04$ ./ftp_client
S: 220 (vsFTPD 3.0.5)
Nhập username: █
```

Ảnh minh họa client nhập tài khoản:

```
sakakiyuya@DESKTOP-7E0C4GH:~/BTVN: 20235367/btvn04$ ./ftp_client
S: 220 (vsFTPD 3.0.5)
Nhập username: user_20235367
C: USER user_20235367
S: 331 Please specify the password.
Nhập password: █
```

Ảnh minh họa client nhập mật khẩu, đăng nhập thành công, tự động tải và up file lên server:

```
sakakiyuya@DESKTOP-7E0C4GH:~/BTVN: 20235367/btvn04$ ./ftp_client
S: 220 (vsFTPD 3.0.5)
Nhap username: user_20235367
C: USER user_20235367
S: 331 Please specify the password.
Nhap password: 536729
C: PASS 536729
S: 230 Login successful.
C: PWD
S: 257 "/" is the current directory
C: PASV
S: 227 Entering Passive Mode (103,101,163,226,39,93).
C: LIST
S: 150 Here comes the directory listing.
S: 226 Directory send OK.

File tren server: question_K4qUsJ.txt

C: PASV
S: 227 Entering Passive Mode (103,101,163,226,39,99).
C: RETR question_K4qUsJ.txt
S: 150 Opening BINARY mode data connection for question_K4qUsJ.txt (101 bytes).

--- NOI DUNG FILE question_K4qUsJ.txt ---
GYPwFX4960aKjS01pnKkb7rtj77TJTgkfcJ2m5YHymm9Vg2snHj4dge5gHRnL29rhEajvuPwWdSAGiF08I3EiSwUgrFQy8L7VS8n
-----

S: 226 Transfer complete.

--- NOI DUNG DA DAO NGUOC ---

n8SV7l8yQFrgUwSiE3I80FiGASdWwPuvjaEhr92lnRHg5egd4jHns2gV9mmyHY5m2JCfkGTJT77jtr7bkKnp10SjKa0694XFWpYG
-----

Da tao file: answer_K4qUsJ.txt

C: PASV
S: 227 Entering Passive Mode (103,101,163,226,40,232).
C: STOR answer_K4qUsJ.txt
S: 150 Ok to send data.
S: 226 Transfer complete.

Upload file answer_K4qUsJ.txt thanh cong!

C: QUIT
S: 221 Goodbye.
```

Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <netdb.h>

#define BUFFER_SIZE 4096

int connect_to_server(const char *ip, int port) {
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) {
        perror("Socket creation failed");
        exit(1);
    }
    struct sockaddr_in server_addr;
    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(port);
    inet_pton(AF_INET, ip, &server_addr.sin_addr);

    if (connect(sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
        perror("Connection failed");
        exit(1);
    }
}
```

```

    }
    return sock;
}

void send_cmd(int sock, const char *cmd) {
    char buffer[BUFFER_SIZE];
    snprintf(buffer, sizeof(buffer), "%s\r\n", cmd);
    send(sock, buffer, strlen(buffer), 0);
    printf("C: %s\n", cmd);
}

void recv_resp(int sock, char *buffer, int size) {
    memset(buffer, 0, size);
    int bytes_received = recv(sock, buffer, size - 1, 0);
    if (bytes_received > 0) {
        printf("S: %s", buffer);
    }
}

void parse_pasv(const char *response, char *ip, int *port) {
    int h1, h2, h3, h4, p1, p2;
    const char *start = strchr(response, '(');
    if (start != NULL) {
        sscanf(start,("(%d,%d,%d,%d,%d,%d)", &h1, &h2, &h3, &h4, &p1, &p2);
        sprintf(ip, "%d.%d.%d.%d", h1, h2, h3, h4);
        *port = (p1 * 256) + p2;
    }
}

void str_reverse(char *str) {
    int i = 0, j = strlen(str) - 1;
    while (i < j) {
        char temp = str[i];
        str[i] = str[j];
        str[j] = temp;
        i++;
        j--;
    }
}

int main() {
    char buffer[BUFFER_SIZE];
    char ip_addr[64];
    int data_port;
    char data_ip[64];
    char filename[256] = {0};

    struct hostent *he = gethostbyname("lebavui.io.vn");
    if (he == NULL) {
        perror("gethostbyname");
    }
}

```

```

        return 1;
    }
    struct in_addr **addr_list = (struct in_addr **)he->h_addr_list;
    strcpy(ip_addr, inet_ntoa(*addr_list[0]));

    int control_sock = connect_to_server(ip_addr, 21);
    recv_resp(control_sock, buffer, BUFFER_SIZE);

    char username[BUFFER_SIZE] = {0};
    char password[BUFFER_SIZE] = {0};

    printf("Nhap username: ");
    fgets(buffer, sizeof(buffer), stdin);
    buffer[strcspn(buffer, "\r\n")] = 0;
    snprintf(username, sizeof(username), "USER %.4090s", buffer);

    send_cmd(control_sock, username);
    recv_resp(control_sock, buffer, BUFFER_SIZE);

    printf("Nhap password: ");
    fgets(buffer, sizeof(buffer), stdin);
    buffer[strcspn(buffer, "\r\n")] = 0;
    snprintf(password, sizeof(password), "PASS %.4090s", buffer);

    send_cmd(control_sock, password);
    recv_resp(control_sock, buffer, BUFFER_SIZE);

    send_cmd(control_sock, "PWD");
    recv_resp(control_sock, buffer, BUFFER_SIZE);

    send_cmd(control_sock, "PASV");
    recv_resp(control_sock, buffer, BUFFER_SIZE);
    parse_pasv(buffer, data_ip, &data_port);

    int data_sock_list = connect_to_server(data_ip, data_port);
    send_cmd(control_sock, "LIST");
    recv_resp(control_sock, buffer, BUFFER_SIZE);

    char list_data[BUFFER_SIZE * 4] = {0};
    int bytes;
    while ((bytes = recv(data_sock_list, buffer, BUFFER_SIZE - 1, 0)) > 0) {
        buffer[bytes] = '\0';
        strcat(list_data, buffer);
    }
    close(data_sock_list);
    recv_resp(control_sock, buffer, BUFFER_SIZE);

    char *ptr = strstr(list_data, "question-");
    if (ptr != NULL) {
        sscanf(ptr, "%255s", filename);
    }

```

```

        printf("\nFile tren server: %s\n\n", filename);
    } else {
        printf("Khong tim thay file question tren server.\n");
        return 1;
    }

    send_cmd(control_sock, "PASV");
    recv_resp(control_sock, buffer, BUFFER_SIZE);
    parse_pasv(buffer, data_ip, &data_port);

    int data_sock_retr = connect_to_server(data_ip, data_port);

    char retr_cmd[512];
    snprintf(retr_cmd, sizeof(retr_cmd), "RETR %s", filename);
    send_cmd(control_sock, retr_cmd);
    recv_resp(control_sock, buffer, BUFFER_SIZE);

    char file_content[BUFFER_SIZE] = {0};
    printf("\n--- NOI DUNG FILE %s ---\n", filename);
    while ((bytes = recv(data_sock_retr, buffer, BUFFER_SIZE - 1, 0)) > 0) {
        buffer[bytes] = '\0';
        printf("%s", buffer);
        strcat(file_content, buffer);
    }
    printf("\n-----\n\n");
    close(data_sock_retr);

    recv_resp(control_sock, buffer, BUFFER_SIZE);

    str_reverse(file_content);
    printf("\n--- NOI DUNG DA DAO NGUOC ---\n%s\n-----\n\n", file_content);

    char answer_filename[256] = {0};
    if (strncmp(filename, "question_", 9) == 0) {
        snprintf(answer_filename, sizeof(answer_filename), "answer_%s", filename +
9);
    } else {
        strcpy(answer_filename, "answer_unknown.txt");
    }

    FILE *f = fopen(answer_filename, "w");
    if (f != NULL) {
        fputs(file_content, f);
        fclose(f);
        printf("Da tao file: %s\n\n", answer_filename);
    }

    send_cmd(control_sock, "PASV");
    recv_resp(control_sock, buffer, BUFFER_SIZE);

```

```
parse_pasv(buffer, data_ip, &data_port);

int data_sock_stor = connect_to_server(data_ip, data_port);

char stor_cmd[512];
snprintf(stor_cmd, sizeof(stor_cmd), "STOR %s", answer_filename);
send_cmd(control_sock, stor_cmd);
recv_resp(control_sock, buffer, BUFFER_SIZE);

send(data_sock_stor, file_content, strlen(file_content), 0);
close(data_sock_stor);

recv_resp(control_sock, buffer, BUFFER_SIZE);
printf("\nUpload file %s thanh cong!\n\n", answer_filename);

send_cmd(control_sock, "QUIT");
recv_resp(control_sock, buffer, BUFFER_SIZE);
close(control_sock);

return 0;
}
```